Alani Architecture

Open Specification v0.1.0-draft — April 2026

This document provides a comprehensive architectural walkthrough of Alani — a bare-metal intelligence operating system that treats reasoning, memory, and decision-making as first-class kernel-managed resources. It is intended for systems researchers, kernel engineers, and AI infrastructure architects who want to understand not just *what* Alani does, but *how* and *why* each architectural decision was made.

1. Architectural Philosophy

Alani is built on a single thesis:

> *"Intelligence should be an operating system primitive — scheduled, isolated, permissioned, and traceable — not an application-layer concern."*

1.1 Why a Kernel?

Application-layer AI orchestration frameworks (LangChain, AutoGen, CrewAI, etc.) treat intelligence as a library concern. They run in userspace, share process memory, lack formal isolation, and offer no system-level governance. This creates fundamental problems:

Problem	Root Cause	Alani's Answer
Context leaks between agents	Shared process memory	Kernel-enforced memory isolation

Problem	Root Cause	Alani's Answer
No resource governance	OS-invisible workloads	Cognitive scheduler with multi-dimensional resource management
Unverifiable decisions	No audit trail	Mandatory trace emission on every syscall
Model coupling	Models embedded in application logic	Models abstracted as swappable devices
No formal security	Trust boundaries at library level	Capability-based access control at syscall boundary

1.2 Why a Microkernel?

Alani follows a microkernel-inspired design rather than a monolithic kernel:

- **Fault isolation** — A failing model device does not crash the kernel. Device failures are contained and reported through the device protocol.
- **Minimized trusted computing base (TCB)** — Only scheduling, IPC, memory management, and cognitive syscall dispatch run in kernel space. Everything else runs in userspace.
- **Extensibility** — New model types, memory backends, and agent runtimes can be added without modifying the kernel.
- **Formal verifiability** — A smaller kernel is more amenable to formal verification, a stated goal for reasoning traces.

1.3 Why Rust?

- **Memory safety without garbage collection** — Critical for a bare-metal kernel that cannot tolerate GC pauses.
- **Ownership model** — Rust's ownership and borrowing system maps naturally to capability-based access control. A capability token is

owned; it can be *borrowed* but not duplicated without explicit kernel action.

- **Zero-cost abstractions** — The device trait system, syscall dispatch, and scheduler policies use Rust's trait and enum systems with no runtime overhead.
- **`no_std` support** — Rust's `no_std` ecosystem enables bare-metal kernel development without a standard library dependency.
- **Fearless concurrency** — The type system prevents data races at compile time.

2. System Topology

Alani implements a five-layer vertically integrated stack. Each layer has a defined trust boundary and communicates with adjacent layers through well-defined interfaces.

Layer 5	**`INTERFACES` Shells, REST APIs, gRPC endpoints, external system bindings** ***Trust: Untrusted***
Layer 4	**`USERSPACE RUNTIME`** Agents, workflows, tool integrations, model hosts *Trust: Capability-bounded*
Layer 3	**`COGNITIVE DEVICES`** Model devices (`/dev/model/*`), Memory devices (`/dev/memory/*`) *Trust: Device-permissioned*
Layer 2	**`ALANI KERNEL (Cognitive Microkernel)`** Scheduler, memory manager, IPC, cognitive syscall dispatch *Trust: Fully trusted (TCB)*
Layer 1	**`HARDWARE`** CPU, RAM, GPU/TPU/NPU, storage, network *Trust: Hardware root of trust*

2.1 Information Flow

The following illustrates the end-to-end information flow for a single intent, from interface receipt through kernel processing to result delivery:

```
Interface ──▶ Userspace Agent ──▶ sys_intent_create() ──▶    Kernel:
Intent Registration     │      ├── sys_context_resolve()  ──▶  Memory Device
(context retrieval)     │      ├── Scheduler: Assign to Model Device     │
├── sys_model_infer()      ──▶  Model Device (inference execution)      │
├── sys_memory_write()     ──▶  Memory Device (result persistence)      │
├── sys_action_execute()   ──▶  Policy Engine (validation + execution)     │
└── sys_trace_emit()       ──▶  Trace Memory (audit log)
```

2.2 Trust Boundaries

Boundary	Location	Enforcement
Syscall boundary	Between userspace and kernel	Capability checks, input validation, permission enforcement
Device boundary	Between kernel and cognitive devices	Device capability negotiation, queue management, fault isolation
Interface boundary	Between external systems and userspace	Input sanitization, authentication, rate limiting

3. The Cognitive Microkernel

The kernel is the trusted core. It governs *how* intelligence operates but never *what* intelligence produces. All probabilistic behavior is confined to device execution outside the kernel's trusted computing base.

3.1 Kernel Subsystems

```
kernel/src/ ├── main.rs              # Entry point, boot sequence ├──
scheduler.rs         # Multi-dimensional cognitive scheduler ├── memory.rs
# Physical and cognitive memory management ├── ipc.rs               # Inter-
process communication ├── syscall.rs           # Syscall dispatch table ├──
device.rs            # Device registry and management ├── capabilities.rs
# Capability token management └── cognition/     ├── mod.rs           #
Cognitive subsystem root     ├── intent.rs        # Intent lifecycle
management     ├── context.rs       # Context resolution engine     ├──
```

```
planner.rs       # Task decomposition and planning     ├── router.rs        #
Model selection and routing     ├── policy.rs        # Governance and policy
enforcement     └── trace.rs         # Trace emission and storage
```

3.2 Kernel Invariants

The following invariants are enforced at all times during kernel execution. Violation of any invariant constitutes a kernel bug.

1. **No probabilistic behavior in kernel space.** The kernel's scheduling, routing, and policy decisions are deterministic given the same inputs. Probabilistic behavior is confined to model device execution in userspace.
2. **No heap allocation on critical paths.** The syscall dispatch, scheduler tick, and IPC message passing operate on pre-allocated structures.
3. **Every syscall emits a trace.** There is no path through the syscall dispatch table that does not produce a trace event. This is enforced by the type system.
4. **Capabilities are unforgeable.** Capability tokens are kernel-allocated, cryptographically tagged integers. Userspace cannot fabricate a valid token.
5. **Intent state transitions are monotonic.** An intent can only move forward through its state machine. No backward transitions are permitted.

3.3 Kernel Data Structures

Intent Table

```
struct IntentTable {     entries: BTreeMap<IntentId, IntentEntry>,
max_entries: usize,     per_agent_quota: usize, }  struct IntentEntry
{     id: IntentId,     owner: AgentId,     state: IntentState,     priority:
u8,     capability: CapabilityToken,     context_ids: Vec<ContextId>,
```

```
created_at: Timestamp,     deadline: Option<Timestamp>,     trace_enabled:
bool, }
```

Device Registry

```
struct DeviceRegistry {     model_devices: BTreeMap<DeviceId,
ModelDeviceHandle>,     memory_devices: BTreeMap<DeviceId,
MemoryDeviceHandle>, }
```

Capability Store

```
struct CapabilityStore {     tokens: BTreeMap<CapabilityToken,
CapabilitySet>, }  struct CapabilitySet {     owner: AgentId,
permissions: BitFlags<Permission>,     namespaces: Vec<Namespace>,
devices: Vec<DeviceId>,     created_at: Timestamp,     expires_at:
Option<Timestamp>, }
```

4. Intent Lifecycle

The intent is the fundamental unit of cognitive work in Alani. Every reasoning task, inference request, and action execution is modeled as an intent that progresses through a well-defined state machine.

4.1 State Machine

The intent state machine defines all valid transitions. States progress monotonically forward; no backward transitions are permitted.

```
┌──────────────────────────────────────────┐                              │
CANCELLED                      │
└──────────────────────────────────────────┘                              ▲
▲         ▲         ▲         ▲                            │         │         │
│         │    ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
┌─────────┐    │ CREATED  ├──▶│ SCHEDULED ├──▶│ RESOLVING ├──▶│ INFERRING
├──▶│ ACTING │    └──────────┘    └──────────┘    └──────────┘    └──────────┘
└─────────┘                                                         │
│                                                   ▼           ▼

┌──────────┐  ┌──────────┐
│ COMPLETED │  │ COMPLETED │
└──────────┘  └──────────┘

┌──────────┐  ┌──────────┐
│  FAILED  │  │  FAILED  │
```

4.2 State Descriptions

State	Description	Kernel Activity
CREATED	Intent registered in the intent table	Capability validated, resources allocated, queued for scheduling
SCHEDULED	Intent assigned to a scheduling slot	Scheduler has committed resources
RESOLVING	Context resolution in progress	Context engine querying memory devices
INFERRING	Model inference in progress	Task dispatched to model device
ACTING	Action execution in progress	Policy engine validated; execution underway
COMPLETED	Intent completed successfully	Results stored, traces finalized, resources released
FAILED	Intent failed	Error recorded, partial results preserved
CANCELLED	Cancelled by caller or policy	In-flight tasks aborted, resources released

4.3 Intent Composition

Intents can be composed hierarchically as a directed acyclic graph (DAG). A parent intent decomposes into child intents that execute independently and report results upward.

```
Parent Intent: "Analyze Q3 financial performance" ├── Child: "Retrieve Q3
revenue data" ├── Child: "Retrieve Q3 expense data" ├── Child: "Compare Q3 to
Q2 trends" └── Child: "Generate executive summary"
```

The kernel enforces the following composition semantics:

- **Lifecycle coupling** — Cancelling a parent cancels all children.
- **Result aggregation** — A parent can wait on all children before proceeding.
- **Resource inheritance** — Children inherit capability tokens (or restricted subsets) from the parent.

- **Trace lineage** — Child traces are linked to the parent, forming a complete causal chain.

5. Context Resolution Engine

The context resolution engine assembles the information needed for a cognitive task. It is the bridge between system memory and a model device's input window, responsible for selecting, prioritizing, and fitting context into the constraints of the target model.

5.1 Resolution Pipeline

Context resolution proceeds through five sequential stages:

Stage	Operation	Description
1	**Explicit Context**	Context items attached by the agent via `sys_context_attach()`
2	**Memory Retrieval**	Queried from memory devices (semantic, structured, key-value)
3	**Intent History**	Prior intents in the same lineage (parent/sibling results)
4	**Policy Filtering**	Remove items the agent lacks permission to access
5	**Window Fitting**	Truncate and prioritize to fit target model's input size

5.2 Context Prioritization

When the assembled context exceeds the target model's input window, items are ranked by the following priority order and lower-priority items are truncated first:

Priority	Source	Rationale
1 (highest)	Explicit context (agent-attached)	Agent's direct intent signal
2	Intent description	The reasoning objective itself
3	Semantic memory (high similarity)	Most relevant retrieved knowledge
4	Structured memory (exact matches)	Factual data matching the query
5	Intent history (parent/sibling)	Surrounding cognitive context
6 (lowest)	Semantic memory (lower similarity)	Broader but less relevant knowledge

5.3 Context Immutability

Once resolved with the `CONTEXT_FLAG_IMMUTABLE` flag, the context snapshot becomes immutable. This provides two critical guarantees:

- **Reproducibility** — The input to the model is deterministic. Given the same immutable context, the same model device will produce the same output (modulo model-internal stochasticity).
- **Auditability** — The exact context that led to a decision is preserved in trace memory and can be replayed for forensic analysis.

6. Cognitive Device Subsystem

Cognitive devices are the computational engines of Alani. They perform actual inference and storage while the kernel governs *when*, *where*, and *under what constraints* they execute.

6.1 Device Taxonomy

```
/dev/ ├── model/ │   ├── llm0            # Large language model │   ├── llm1
# Second LLM instance │   ├── vision0         # Vision model │   ├── embed0
# Embedding model │   ├── speech0         # Speech recognition │   └── code0
# Code generation └── memory/     ├── vector0         # Vector similarity
store     ├── graph0          # Knowledge graph     ├── kv0             #
Key-value store     └── doc0            # Document store
```

6.2 Device Protocol

Every cognitive device implements a common trait protocol. This uniform interface allows the kernel to manage heterogeneous devices through a single abstraction:

```
trait CognitiveDevice {     fn capabilities(&self) -> DeviceCapabilities;
fn submit(&mut self, task: DeviceTask) -> Result<TaskHandle, DeviceError>;
fn poll(&self, handle: TaskHandle) -> TaskStatus;     fn cancel(&mut self,
handle: TaskHandle) -> Result<(), DeviceError>;     fn queue_status(&self) ->
QueueStatus;     fn health(&self) -> Result<(), DeviceError>; }
```

6.3 Device Capabilities

Each device declares its capabilities at registration time. The kernel uses this metadata for routing, scheduling, and admission control:

```
struct DeviceCapabilities {     device_type: DeviceType,     modalities:
Vec<Modality>,       // Text, Image, Audio, Code, Embedding
max_input_bytes: usize,     max_output_bytes: usize,     max_queue_depth:
usize,     supports_streaming: bool,     supports_batching: bool,
estimated_latency_ms: u64,     cost_per_token: Option<f64>, }
```

6.4 Device Lifecycle

```
┌──────────┐    ┌──────────┐    ┌─────────┐    │ DISCOVERED │──▶│
REGISTERED │──▶│ ACTIVE │    └──────────┘    └─────────┘
└────┬─────┘                                  │              │
▼                ▼                                ┌─────────┐    ┌─────────┐
│ REJECTED │     │ DEGRADED │                                └─────────┘
└──────┘                                                      │
▼                                                ┌────────┐
│ ACTIVE │ (recovery)                                        └────────┘
```

6.5 Model Routing

When a syscall specifies `device_id = 0` (auto-select), the kernel router selects the optimal device using the following algorithm:

6. **Filter** — Exclude devices lacking required capabilities (modality, input size, health status).
7. **Rank** — Score remaining devices by: queue depth, estimated latency vs. intent deadline, cost per token, and health status.
8. **Select** — Choose the highest-ranked device.
9. **Fallback** — If no device qualifies, return `-ENODEV`.

7. Memory Architecture

Alani's memory architecture is organized into four tiers, each with distinct access controls, persistence guarantees, and intended use cases.

7.1 Memory Tiers

Tier 1	**`KERNEL COGNITIVE MEMORY`** ***Contents:*** **Policy tables, capability store, intent table, device registry** ***Access:*** **Kernel only. No userspace access.** ***Persistence:*** **Survives agent restarts. Lost on reboot unless backed.**
Tier 2	**`USER COGNITIVE MEMORY`** *Contents:* Per-agent reasoning state, working memory, scratchpad *Access:* Owning agent only. Isolated by capability. *Persistence:* Lifetime of the agent or intent.
Tier 3	**`SHARED SEMANTIC MEMORY`** *Contents:* Embeddings, documents, cross-agent shared knowledge *Access:* Namespace-permissioned via capabilities. *Persistence:* Permanent unless TTL-expired or explicitly deleted.
Tier 4	**`EXECUTION TRACE MEMORY`** *Contents:*

Tier 1	**`KERNEL COGNITIVE MEMORY`** ***Contents:*** **Policy tables, capability store, intent table, device registry** ***Access:*** **Kernel only. No userspace access.** ***Persistence:*** **Survives agent restarts. Lost on reboot unless backed.**
	Syscall traces, intent lifecycle events, decision audit trails *Access:* `CAP_TRACE_READ` holders. Append-only. *Persistence:* Permanent. Immutable after write.

7.2 Memory Namespaces

```
/memory/ ├── global/                    # System-wide shared knowledge ├── research/                  # Research domain ├── operations/                # Operational data ├── agent.<agent_id>/          # Per-agent private namespace └── intent.<intent_id>/        # Per-intent ephemeral namespace
```

7.3 Memory Operations by Device Type

Device Type	Operations	Use Case
Vector store (`/dev/memory/vector0`)	Write: embed + index; Query: similarity search	Semantic retrieval, RAG
Knowledge graph (`/dev/memory/graph0`)	Write: add node/edge; Query: graph traversal	Relational reasoning, entity relationships
Key-value store (`/dev/memory/kv0`)	Write: set; Query: get, scan	Configuration, session state
Document store (`/dev/memory/doc0`)	Write: index document; Query: full-text search	Document retrieval, corpus search

7.4 Memory Consistency Model

- **Within an intent:** Linearizable. A write is visible to all subsequent reads in the same intent.
- **Across intents:** Eventually consistent by default, unless `MEMORY_FLAG_SEMANTIC` forces synchronous indexing.

- **Trace memory:** Append-only, immutable, ordered by kernel timestamp.

8. Cognitive Scheduler

The cognitive scheduler is the most architecturally novel component of Alani. Traditional OS schedulers manage one dimension (CPU time). Alani's scheduler manages six.

8.1 Scheduling Dimensions

Dimension	Description
CPU Time	Standard time-slice scheduling for agent execution
Memory Bandwidth	Rate of memory device read/write operations
Model Slots	Concurrent inference tasks per model device
Context Window	Aggregate context size across all active intents
Latency Budget	Remaining time before intent deadline
Cost Budget (optional)	Token/compute cost budget for distributed inference

8.2 Scheduling Algorithm

The scheduler uses a multi-resource fair queuing approach. On each scheduler tick, the following stages execute in order:

```
Score = (priority_weight × priority)       + (deadline_weight × urgency)
+ (efficiency_weight × estimated_cost_inverse)  where urgency = 1 / (deadline
− current_time)
```

8.3 Preemption

- **Priority preemption** — A higher-priority intent preempts a lower-priority intent's model device slot (only if inference has not yet begun).
- **Deadline preemption** — An intent approaching its deadline can preempt lower-priority intents to meet timing constraints.
- **Policy preemption** — The policy engine can force-cancel violating intents at any time.

8.4 Backpressure

When the system is under load, the scheduler applies backpressure through four mechanisms:

10. **Queue depth limiting** — Each device has a maximum queue depth; submissions beyond this limit return `-EBUSY`.
11. **Admission control** — Intents are held in `CREATED` state until sufficient resources become available.
12. **Intent quota** — Per-agent maximum concurrent intents, enforced at `sys_intent_create` with `-EQUOTA`.
13. **Graceful degradation** — Route to lower-capability but available devices when primary devices are saturated.

9. Policy Engine

The policy engine is the governance layer of Alani. It determines what actions are permitted, under what conditions, and with what constraints. Policy evaluation is embedded in the syscall dispatch path — there is no way to bypass it.

9.1 Policy Evaluation Pipeline

```
┌──────────────────────┐   │  1. Capability      │  Does the agent's token
include   │     Check           │  the required permission?
└──────────────────────┘            ▼   ┌──────────────────────┐   │  2.
Policy Rule     │  Do active rules permit this   │     Evaluation      │
action type + payload?   └──────────────────────┘             ▼
┌──────────────────────┐   │  3. Confirmation    │  Does the policy require
│     Gate             │  user confirmation?   └──────────────────────┘
▼   ┌──────────────────────┐   │  4. Constraint      │  Apply rate limits,
output   │     Application     │  filters, audit requirements
└──────────────────────┘            ▼   ┌──────────────────────┐   │  Action
Executed    │   └──────────────────────┘
```

9.2 Policy Rules

```
struct PolicyRule {     id: PolicyRuleId,     action_type: ActionTypePattern,
// e.g., "email.*", "file.delete"     agent_pattern: AgentPattern,         //
e.g., "*", "agent.research.*"     effect: PolicyEffect,                //
Allow, Deny, RequireConfirmation     constraints: Vec<PolicyConstraint>,
priority: u16, }  enum PolicyEffect {     Allow,     Deny { reason: &'static
str },     RequireConfirmation { prompt: &'static str }, }  enum
PolicyConstraint {     RateLimit { max_per_minute: u32 },     MaxPayloadSize
{ bytes: usize },     RequireTrace,     RequireReview { reviewer:
AgentId }, }
```

9.3 Default Policies

Policy	Effect	Rationale
All syscalls require valid capability token	Deny if missing	Baseline security — no anonymous syscalls
All actions require trace	RequireTrace	Auditability — no untraced operations
External actions (email, API calls)	RequireConfirmation	Prevent unintended side effects
Memory writes to `global/` namespace	Deny for unprivileged agents	Protect shared knowledge base
Model inference	Allow for `CAP_MODEL_INFER` holders	Standard operation — permissioned access
Intent creation beyond quota	Deny with `-EQUOTA`	Resource protection — prevent exhaustion

10. Security Architecture

10.1 Threat Model

Threat	Vector	Mitigation
Agent privilege escalation	Forged capability token	Kernel-allocated, cryptographically tagged tokens
Cross-agent memory access	Direct memory read	Namespace isolation, capability-gated access
Model device hijacking	Unauthorized inference	Device-level permission checks at submission
Trace tampering	Modify audit log	Append-only trace memory, kernel-only writes
Policy bypass	Skip policy check	Policy evaluation embedded in syscall dispatch
Resource exhaustion	Unbounded intent creation	Per-agent quotas, admission control
Context injection	Malicious context attachment	Policy filtering in context resolution pipeline

10.2 Capability Architecture

Capabilities are the foundation of Alani's security model. They replace traditional access control lists with a more composable and delegatable mechanism:

- **Kernel-allocated** — Only the kernel can create capability tokens. There is no userspace API for token creation.
- **Unforgeable** — Tokens are cryptographically tagged. Userspace cannot fabricate a valid token.
- **Fine-grained** — Permissions are specified per-syscall, per-namespace, per-device.

- **Delegatable** — An agent can create a restricted subset of its own capabilities for child intents.
- **Revocable** — The kernel can revoke capabilities at any time, immediately invalidating all operations using that token.
- **Time-bounded** — Capabilities can have expiration timestamps, after which they are automatically invalid.

10.3 Isolation Boundaries

```
┌──────────────────────────────────────────────────────────────────────┐  │  Agent A
(Capability: CAP_A)                                   │ │
│ │  Can access:                                                        │ │
/memory/agent.A/*                                                     │ │
/dev/model/llm0                                                       │ │
│ │  Cannot access:                                                     │ │
/memory/agent.B/*                                                     │ │
/dev/model/vision0                                                    │
└──────────────────────────────────────────────────────────────────────┘
┌──────────────────────────────────────────────────────────────────────┐  │  Agent B
(Capability: CAP_B)                                   │ │
│ │  Can access:                                                        │ │
/memory/agent.B/*                                                     │ │
/memory/global/  (read only)                                          │ │
/dev/model/llm0                                                       │ │
/dev/model/vision0                                                    │ │
│ │  Cannot access:                                                     │ │
/memory/agent.A/*                                                     │
└──────────────────────────────────────────────────────────────────────┘  Both agents
invoke the same syscalls, but the kernel enforces different permissions based
on their capability tokens.
```

11. Observability and Tracing

11.1 Trace Architecture

Every syscall emits a structured trace event. This is mandatory — there is no opt-out. Trace events are:

- **Structured** — JSON-serializable with a defined schema.
- **Timestamped** — Kernel-assigned microsecond timestamps.

- **Linked** — Every trace is linked to an `intent_id`, creating causal chains.
- **Immutable** — Written to append-only trace memory.
- **Queryable** — Accessible via `sys_trace_query` for holders of `CAP_TRACE_READ`.

11.2 Trace Event Schema

Field	Type	Description
`intent_id`	`uint64`	Associated intent
`syscall`	`string`	Syscall name
`timestamp_us`	`uint64`	Kernel timestamp in microseconds
`caller`	`AgentId`	Agent that invoked the syscall
`duration_us`	`uint64`	Execution duration in microseconds
`result`	`int`	Return code (0 = success, negative = error)
`device_id`	`uint32`	Device used (if applicable)
`metadata`	`object`	Syscall-specific additional data

11.3 Trace Queries

The following is an example trace for a complete intent lifecycle, queried via `sys_trace_query(intent_id=8472)`:

```
{   "intent_id": "8472",   "events": [     {       "syscall":
"sys_intent_create",       "timestamp_us": 1000000,       "result": 0,
"duration_us": 42     },     {       "syscall": "sys_context_resolve",
"timestamp_us": 1000200,       "result": 0,       "duration_us": 3200,
"device_id": "vector0"     },     {       "syscall": "sys_model_infer",
"timestamp_us": 1004000,       "result": 0,       "duration_us": 842000,
"device_id": "llm0"     },     {       "syscall": "sys_memory_write",
"timestamp_us": 1846500,       "result": 0,       "duration_us": 150     },
{       "syscall": "sys_action_execute",       "timestamp_us": 1847000,
```

```
"result": 0,       "duration_us": 50     }   ],   "total_latency_us": 847092,
"models_used": ["llm0"],   "memory_devices_used": ["vector0"],   "status":
"completed" }
```


11.4 Observability Use Cases

Use Case	Method	Value
Decision audit	Trace query by `intent_id`	Complete chain from intent creation to action execution
Performance analysis	Aggregate latency metrics across intents	Identify bottlenecks in model inference or context resolution
Cost accounting	Sum token usage per agent	Budget enforcement and chargeback
Security forensics	Filter traces by agent or device	Detect unauthorized access attempts or anomalous patterns
Debugging	Intent lifecycle replay	Reproduce and diagnose failures with full context

12. Userspace Runtime

12.1 Agent Model

An agent is the primary unit of cognitive execution in userspace. Agents:

- Hold a capability token granting specific permissions.
- Create intents via `sys_intent_create`.
- Execute within a sandboxed environment.
- Communicate with other agents only through kernel-mediated IPC.
- Are lifecycle-managed by the kernel (creation, suspension, termination).

12.2 Agent Structure

```
struct Agent {     id: AgentId,     capability: CapabilityToken,     state:
AgentState,     active_intents: Vec<IntentId>,     memory_namespace:
Namespace,     ipc_channels: Vec<ChannelId>, }
```

12.3 Agent Types

Type	Description	Example
Reactive	Responds to external events	API request handler
Autonomous	Pursues goals independently	Research agent
Orchestrator	Coordinates other agents	Workflow manager
Tool	Provides a specific capability	Calculator, web search wrapper

12.4 Userspace Components

```
userspace/ ├── agents/              # Agent runtime and lifecycle ├── tools/
# Tool integrations ├── shell/               # Interactive cognitive shell
├── workflows/           # Workflow definition and execution ├── model_hosts/
# Model device host processes └── memory_hosts/        # Memory device host
processes
```

13. Inter-Process Communication

13.1 IPC Mechanisms

Alani provides kernel-mediated IPC through four mechanisms:

Mechanism	Description	Use Case
Message passing	Typed, buffered messages between agents	Agent-to-agent communication
Shared memory	Kernel-managed shared memory regions	High-throughput data sharing
Channels	Bidirectional, typed	Structured agent

Mechanism	Description	Use Case
	communication channels	conversations
Signals	Lightweight, asynchronous notifications	Event notification

13.2 IPC Security

All IPC is capability-gated:

- An agent can only send messages to agents for which it holds a valid communication capability.
- Shared memory regions are created with explicit capability grants specifying read, write, or read-write access.
- The kernel logs all IPC events to trace memory for full traceability.

14. Boot Sequence

14.1 Boot Stages

14. **Hardware initialization** — CPU, memory, interrupt controllers.
15. **Kernel initialization** — Scheduler, memory manager, IPC subsystem.
16. **Cognitive subsystem initialization** — Intent table, device registry, capability store, policy engine, trace system.
17. **Device discovery** — Scan for model and memory devices, validate capabilities, register in device registry.
18. **Userspace initialization** — Load agent runtime, start shell (if configured).
19. **Ready** — System accepts intents.

14.2 Boot Trace

The boot sequence itself is traced, producing a boot trace that records initialization timing and device discovery results. This trace is written to trace memory as the first entries, providing a complete record of system startup for diagnostics and performance analysis.

15. Error Model

15.1 Error Propagation

Errors propagate upward through the stack in a structured manner:

```
Device Error ──▶ Kernel wraps as syscall error code ──▶ Userspace agent
handles or propagates
```

Every error is captured in the trace system, ensuring that even transient failures are recorded for later analysis.

15.2 Error Categories

Category	Examples	Recovery
Permission errors	`-EPERM`	Agent requests additional capabilities
Resource errors	`-ENOMEM`, `-EQUOTA`, `-EBUSY`	Retry with backoff, reduce load
Device errors	`-ENODEV`, `-EFAULT`	Route to alternate device
Validation errors	`-EINVAL`, `-E2BIG`	Fix input and retry
Timeout errors	`-ETIMEOUT`	Retry or cancel intent
Policy errors	`-EPOLICY`	Obtain user confirmation or adjust action

15.3 No Panics

The kernel never panics. All error conditions are handled through Rust's `Result` types. Device failures are contained — a crashed model device does not bring down the kernel or affect other devices. This is a direct benefit of the microkernel architecture described in Section 1.2.

16. Concurrency Model

16.1 Kernel Concurrency

- The kernel uses a multi-core aware design with per-CPU run queues.
- Kernel data structures are protected by fine-grained locks (spinlocks for short critical sections, mutexes for longer operations).
- The intent table, device registry, and capability store each have independent locks to minimize contention.
- Rust's ownership system enforces lock discipline at compile time, preventing data races.

16.2 Agent Concurrency

- Each agent runs in its own execution context (analogous to a process).
- Multiple intents from the same agent may execute concurrently on different model devices.
- Syscalls within a single intent are serialized by the kernel.
- Syscalls across different intents may execute concurrently.

16.3 Device Concurrency

- Each model device maintains its own task queue.

- The device implementation determines internal parallelism (e.g., batched inference).
- The kernel does not dictate device-internal concurrency — it only manages the queue boundary.

17. Hardware Abstraction

17.1 HAL Design

The Hardware Abstraction Layer provides a uniform interface across hardware platforms, isolating the kernel from platform-specific details:

```
trait HardwareAbstraction {     fn cpu_count(&self) -> usize;     fn
memory_size(&self) -> usize;     fn accelerators(&self) ->
Vec<AcceleratorInfo>;     fn timer(&self) -> &dyn Timer;     fn
interrupt_controller(&self) -> &dyn InterruptController; }
```

17.2 Accelerator Support

Accelerator	Role	Integration
GPU	Model inference	Exposed as model device backend
TPU	High-throughput inference	Exposed as model device backend
NPU	Edge inference	Exposed as model device backend
FPGA	Custom inference pipelines	Custom device driver

Accelerators are not accessed directly by the kernel — they are managed by userspace device host processes that implement the `CognitiveDevice` trait. This design ensures that accelerator driver bugs cannot compromise the kernel.

18. Extension Points

Alani is designed for extensibility at every layer without requiring kernel modification.

18.1 Custom Model Devices

Any process that implements the `CognitiveDevice` protocol can register as a model device:

- Local LLM inference engines (llama.cpp, vLLM)
- Remote API wrappers (OpenAI, Anthropic, Google)
- Custom models (fine-tuned, domain-specific)
- Ensemble models (multi-model voting)

18.2 Custom Memory Devices

Any storage backend that implements the memory device protocol:

- Vector databases (Qdrant, Milvus, Pinecone)
- Graph databases (Neo4j, TigerGraph)
- Traditional databases (PostgreSQL, SQLite)
- Custom stores (in-memory, distributed)

18.3 Custom Agents

The agent runtime supports custom agent types:

- Single-purpose tool agents
- Multi-step reasoning agents
- Human-in-the-loop agents
- Federated agents (spanning multiple Alani instances)

18.4 Custom Policies

The policy engine accepts custom policy rules at runtime, enabling:

- Organization-specific governance
- Compliance requirements (GDPR, HIPAA)
- Cost management policies
- Safety constraints

19. Comparison with Existing Architectures

19.1 Architectural Comparison

Dimension	Traditional OS (Linux)	AI Orchestration Frameworks	Cloud AI Platforms	Alani
Intelligence location	None — applications only	Application library	Managed service	Kernel layer
Model abstraction	N/A	Library objects, API clients	Service endpoints	Kernel-managed devices
Memory isolation	Process-level	None or manual	Service-level	Namespace-isolated, capability-gated
Scheduling	CPU time only	None	Queue-based	Multi-dimensional (6 dimensions)
Security model	User/group (DAC)	API keys	IAM policies	Capability-based (per-syscall)
Observability	System logs	Application logs	Service metrics	Mandatory kernel-level tracing

Dimension	Traditional OS (Linux)	AI Orchestration Frameworks	Cloud AI Platforms	Alani
Auditability	Limited (audit subsystem)	None	Cloud trail	Every intent traceable end-to-end
Fault isolation	Process boundaries	None (shared memory)	Service boundaries	Device + agent sandboxing
Formal governance	N/A	None	Policy as code	Kernel-enforced policy engine

19.2 Analogies to Existing OS Concepts

Traditional OS Concept	Alani Equivalent
Process	Agent
System call	Cognitive syscall
File descriptor	Intent ID
Device driver	Cognitive device driver
Filesystem	Memory namespace
Scheduler	Cognitive scheduler (multi-dimensional)
Security module (SELinux)	Policy engine
Audit subsystem	Trace system
IPC (pipes, sockets)	Agent-to-agent channels

20. Design Tradeoffs

20.1 Explicit Tradeoffs

Tradeoff	Choice	Alternative	Rationale
Microkernel vs. monolithic	Microkernel	Monolithic kernel	Fault isolation and formal verifiability

Tradeoff	Choice	Alternative	Rationale
			outweigh IPC overhead. Model device crashes must not bring down the kernel.
Rust vs. C	Rust	C (traditional kernel language)	Memory safety is non-negotiable for a system managing cognitive workloads. The cost is ecosystem maturity.
Capability-based vs. ACL	Capability-based	Access control lists	Capabilities are more composable and delegatable, essential for dynamic agent ecosystems. ACLs scale poorly with agent proliferation.
Deterministic kernel vs. adaptive	Deterministic	Adaptive/learning kernel	The kernel must be auditable and predictable. Adaptive behavior belongs in userspace agents, not the TCB.
Mandatory tracing vs. optional	Mandatory	Opt-in tracing	Auditability is a first-class requirement, not a debugging convenience. The overhead is acceptable for the guarantee.
Eventual consistency vs. strong	Eventual (cross-intent)	Strong consistency	Strong consistency across all intents would serialize the entire system. Eventual consistency allows concurrent cognitive workloads.

20.2 Open Questions

20. **Distributed cognitive kernels** — How should multiple Alani instances coordinate? Federated capability delegation? Distributed intent scheduling?
21. **Formal verification scope** — Which kernel invariants should be formally verified first? Intent state machine? Capability system? Policy engine?
22. **Cognitive syscall ABI stability** — When should the syscall interface be frozen for backwards compatibility?
23. **Hardware-specific optimization** — How deeply should the kernel integrate with GPU/TPU/NPU scheduling, vs. delegating entirely to device drivers?
24. **Real-time guarantees** — Should Alani provide hard real-time guarantees for time-critical cognitive workloads?

References

[1] A. S. Tanenbaum, *Modern Operating Systems*, 4th ed. Pearson, 2014.

[2] J. Liedtke, "On micro-kernel construction," in *Proc. 15th ACM SOSP*, 1995, pp. 237–250.

[3] R. Klabnik and S. Nichols, *The Rust Programming Language*. No Starch Press, 2019.

[4] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017.

[5] J. S. Chase et al., "Sharing and protection in a single-address-space operating system," *ACM TOCS*, vol. 12, no. 4, 1994.

[6] D. R. Engler, M. F. Kaashoek, and J. O'Toole Jr., "Exokernel: An operating system architecture for application-level resource management," in *Proc. 15th ACM SOSP*, 1995, pp. 251–266.

[7] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proc. IEEE S&P*, 1987.

[8] M. D. Schroeder and J. H. Saltzer, "The protection of information in computer systems," in *Proc. IEEE*, vol. 63, no. 9, 1975.

[9] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[10] Y. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

Intelligence is no longer an application feature — it is an operating system primitive.